

A100 Strong Scaling 개선 작업지시서 v2

PaScaL Poisson FFT GPU vs CaNS
교정된 소스 감사와 단계별 구현 명세

대상	KISTI Neuron / NVIDIA A100 / NVHPC 25.11 / CUDA 12.9.1
고정 조건	256^3, FP64, cube MPI decomposition 유지
정확성	solution dump 기본 OFF, 마지막 해 RMS 1회, solve timer 밖
현재 산출물	Stage 1 C2J/J2C 상세 route profiler 구현본 포함
작성일	2026-07-29

판정 원칙: 소스 구조로 가설을 세우고, warmed debug profile로 원인을 분해한 뒤, 성능 빌드의 rank-max 중앙값으로만 채택 여부를 결정한다.

0. 한 장 결론

현재 코드 상태를 반영해 우선순위를 다시 정리한 실행 요약

현재 1순위는 새 통신 라이브러리를 바로 붙이는 것이 아니라 C2J/J2C의 pack, MPI, unpack 비중을 분리해서 Stage 2의 구현 방향을 증거로 결정하는 것이다. 이 계측은 이번 ZIP에 이미 구현되어 있다.

현재 v2에서 이미 해결된 항목

- CUDA-aware MPI와 cuFFT의 첫 호출 비용을 strong-scaling 시간에서 제외: debug warm-up + perf 5회 warm-up/20회 평균.
- debug와 performance 빌드를 한 번의 make로 함께 생성하고 모듈/오브젝트 디렉터리를 분리.
- FFT count/displacement 메타데이터, MPI device buffer, TDMA workspace, ghost buffer, cuFFT plan을 반복 solve에서 재사용.
- 평균 제거의 세 방향 연속 Allreduce를 MPI_COMM_WORLD 단일 Allreduce로 변경.
- solution dump는 기본 OFF. 마지막 solution의 RMS만 한 번 계산하며 모든 solve/batch timer 밖에 둬.

이번 Stage 1 구현

항목	구현 내용	성능 빌드 영향
C2J / J2C	pack + MPI 전 dependency wait / MPI_Alltoallv / unpack + 완료 wait 분리	없음
데이터량	rank별 complex payload MiB 계산	없음
대역폭	payload / route total의 rank-min/avg/max GB/s	없음
검증	debug/perf 전처리 분리, shell 문법, make dry-run	실컴파일은 Neuron에서 수행

중요한 교정: CaNS가 모든 TDMA factor를 정상상태에서 완전히 캐시한다고 단정할 수 없다. 첨부 소스는 aa_z_save/cc_z_save로 일부 전치된 경계 계수를 재사용하지만 local elimination과 reduced solve 작업은 여전히 수행한다. 따라서 PaScaL의 Stage 5도 실제 불변성과 in-place 수정 여부를 증명한 배열만 캐시해야 한다.

1. 측정 계약과 현재 기준선

구현 전후 비교에서 절대 바꾸지 않을 조건

1.1 고정 조건

- 전역 격자 $256 \times 256 \times 256$, FP64, 같은 경계조건, 같은 해석해.
- 사용자가 요구한 cube형 MPI decomposition을 유지. 자동 pencil 재배포는 금지.
- NP=1,2,4부터 검증하고 이후 8,16,32로 확대. 최종 비교는 가능한 한 동일 partition과 유사한 배치에서 반복.
- 강한 확장성 시간은 performance build의 Steady-state average rank-max만 사용.
- RHS 재생성, final D2H, RMS reduction, 파일 출력은 solve timer에서 제외.

1.2 현재 관측 기준선

GPU	분해	rank-max ms/solve	NP1 대비 speedup	효율
1	1 x 1 x 1	5.798196	1.000	100.0%
2	1 x 1 x 2	4.970802	1.166	58.3%
4	1 x 2 x 2	7.187008	0.807	20.2%

NP=4 warmed debug에서 C2J max 1.978780 ms, J2C max 2.123600 ms로 두 route 합계가 약 4.10238 ms였다. 이는 7.187008 ms solve의 약 57%다. 다만 기존 whole-route timer만으로는 MPI 자체와 GPU pack/unpack 중 어느 쪽이 지배적 인지 분리할 수 없었다.

1.3 채택 기준

검사	최소 조건	권장 조건
정확성	모든 NP/build에서 final RMS 일치	$\text{abs}(\text{delta RMS}) \leq \max(1e-12, 1e-8 \times \text{abs}(\text{base RMS}))$
성능	NP1 회귀 $\leq 3\%$	목표 NP의 rank-max 중앙값 $\geq 10\%$ 개선
반복성	NP당 독립 job 3회	중앙값과 min/max 함께 기록
출력	P_*.PLT 생성 금지	debug/perf log와 RMS만 보존

같은 노드에서 실행한 것은 중요하다. NP=1,2,4가 모두 A100 8장이 연결된 동일 노드 안에 있으면 노드 간 네트워크 변동을 제거할 수 있다. 그래도 동시에 다른 job이 같은 node/interconnect를 점유할 수 있으므로 최종 표는 MAX_ACTIVE_JOBS=1과 3회 중앙값으로 확정한다.

2. CaNS에서 실제로 배울 것과 배우지 말 것

첨부 CaNS commit f3019f93... 및 bundled cuDecomp 코드 기준

CaNS 소스에서 확인된 사실	PaScaL 적용 의미	주의
GPU 선택 후 cuDecomp grid 생성	rank-GPU binding을 backend 초기화보다 먼저 고정	현재 mapping log로 계속 검증
transpose/halo backend autotuning	MPI/NCCL 후보를 같은 입력에서 비교할 수 있음	첨부 1 x 1 결과는 multi-GPU 증거가 아님
cuFFT auto allocation OFF + shared work area	plan별 workspace 중복과 allocator 변동을 줄일 수 있음	Stage 7에서 stream과 함께 적용
OpenACC queue 1 CUDA stream을 취득	통신/FFT/커널의 dependency를 한 stream으로 표현	CUDA Fortran kernel까지 같은 stream이어야 안전
PCR-TDMA용 별도 reduced grid	큰 transpose와 작은 boundary transpose를 구분	현재 PaScaL layout과 직접 호환된다는 보장은 없음
aa_z_save / cc_z_save	불변 경계계수 일부 재사용 가능성	전체 factor 캐시로 과장 금지

교정된 통신 구조 해석

- CaNS 일반 FFT Poisson 경로는 대표적으로 네 번의 full pencil transpose를 가진다.
- CaNS PCR-TDMA 경로는 두 번의 full transpose와 두 번의 reduced boundary transpose를 사용한다.
- PaScaL의 복소 TDMA가 real/imag를 별도로 처리하고 각 core에서 여러 계수/RHS transpose를 수행한다는 점은 강한 최적화 후보다.
- 그러나 CaNS의 reduced layout을 그대로 이식하는 것은 Stage 2가 아니다. 먼저 기존 buffer layout을 유지한 NCCL backend가 더 작은 변경이다.

금지되는 성급한 결론

NCCL이 항상 MPI보다 빠르다, cuDecomp를 붙이면 자동으로 빨라진다, stream sync를 지우면 overlap이 생긴다, TDMA 계수는 전부 불변이라는 가정은 모두 금지한다. 각 주장은 해당 Stage의 profile과 RMS 결과로 입증한 뒤 채택한다.

우선순위가 달라진 이유

현재 v2는 hot path allocation과 plan 재생성, 평균 제거 collective를 이미 개선했다. 따라서 예전 작업지시서의 P0 일부는 완료 상태이며, 실제 다음 P0는 4 GPU에서 solve의 절반 이상을 차지한 C2J/J2C route를 정확히 분해하는 것이다.

3. Stage 1 - C2J/J2C 상세 route profile

상태: 이번 ZIP에 구현 완료, Neuron 실행 대기

3.1 계측 경계

구간	시작	종료	의미
pack + dependency wait	첫 peer pack kernel enqueue 직전	기존 cudaStreamSynchronize 직후	GPU pack 실행과 MPI 안전성에 필요한 wait
MPI_Alltoallv	dependency wait 직후	blocking collective 반환 직후	CUDA-aware MPI 통신/collective 지연
unpack + completion wait	첫 unpack enqueue 직전	debug-only stream sync 직후	GPU unpack 완료 시간

3.2 출력

```
===== FFT route detail =====
[ROUTE] C2J pack + dependency wait [s]      min avg max
[ROUTE] C2J MPI_Alltoallv [s]              min avg max
[ROUTE] C2J unpack + completion wait [s]    min avg max
[ROUTE] C2J route total [s]                min avg max
[ROUTE] C2J payload sent / rank [MiB]       min avg max
[ROUTE] C2J effective payload BW [GB/s]     min avg max
... J2C ...
=====
```

3.3 구현 안전성

- 모든 내부 타이머와 debug-only unpack sync는 POISSON_DETAILED_PROFILE 내부에만 존재.
- performance 전처리 결과에는 route_t0, route_phase_times, 상세 report 호출이 남지 않음.
- 기존 MPI 전 cudaStreamSynchronize는 correctness dependency이므로 유지.
- rank별 payload를 route total로 나눈 diagnostic bandwidth를 출력. 링크의 이론 대역폭으로 해석하지 않음.
- NP=1은 inter-rank route가 없으므로 No inter-rank exchange 메시지가 정상.

3.4 Stage 2 선택 게이트

관측 결과	다음 행동
MPI >= route의 50% 또는 두 route MPI 합계 >= solve의 30%	Stage 2 NCCL grouped P2P backend
pack + unpack >= route의 25% 또는 합계 >= 0.5 ms	Stage 3 one-shot pack/unpack을 Stage 2와 병행 비교
C2J + J2C < solve의 20%	route 최적화 중단, Stage 4 TDMA profile로 이동

Stage 1 debug 숫자는 원인 분해용이다. 강한 확장성 그래프에 넣는 시간은 반드시 performance build의 warmed repeated-solve rank-max다.

4. Stage 2 - 선택형 NCCL route backend

cube decomposition과 기존 pack layout을 유지하는 최소 변경

4.1 설계

- 기존 `fft_meta_c2j_complex` / `fft_meta_j2c_complex`의 counts와 displacements를 그대로 사용.
- 각 MPI communicator의 rank 목록으로 NCCL communicator를 init 단계에서 한 번 생성하고 cleanup에서 해제.
- Alltoallv를 `ncclGroupStart`, peer별 `ncclSend/ncclRecv`, `ncclGroupEnd`로 치환.
- self peer는 device copy 또는 기존 buffer slice 유지. zero count peer는 호출 생략.
- 환경변수 `POISSON_FFT_ROUTE_BACKEND=mpi|nccl`로 런타임 선택. 기본값은 `mpi`.
- 초기 버전은 blocking-equivalent dependency로 구현하고 correctness를 고정한 뒤 overlap을 검토.

4.2 필요한 코드 경계

영역	추가/변경
module state	C2J/J2C communicator별 <code>ncclComm</code> , CUDA stream handle, initialized flag
plan create	MPI rank mapping 수집, <code>ncclUniqued</code> broadcast, communicator 생성
route exchange	기존 <code>MPI_Alltoallv</code> 와 NCCL grouped P2P backend 분기
cleanup	<code>ncclCommDestroy</code> , stream ownership 규칙에 따른 해제
Makefile	<code>-cudalib=nccl</code> 또는 실제 Neuron link 경로를 <code>dryrun</code> 으로 검증

4.3 채택/롤백

검사	채택 기준	실패 시
RMS	MPI backend와 허용오차 내 일치	즉시 rollback
NP=1	회귀 $\leq 3\%$	self path 수정 또는 NCCL 비활성
NP=2/4	3회 중앙값 $\geq 10\%$ 개선	MPI 유지, Stage 3 또는 TDMA 이동
확장	NP=8 이상에서도 오류/교착 없음	communicator mapping과 zero-count 점검

NCCL backend는 direct replacement 실험이다. 이 단계에서 decomposition, FFT layout, TDMA 알고리즘 까지 함께 바꾸면 원인 추적과 rollback이 불가능해진다.

5. Stage 3 - pack/unpack launch 수 축소

Stage 1에서 GPU 재배포 비용이 실제로 보일 때만 수행

5.1 현재 위험 신호

현재 C2J/J2C는 peer별로 pack kernel을 한 번씩 launch하고 peer별 unpack kernel을 다시 launch한다. GPU 수가 증가하면 각 kernel의 데이터량은 작아지고 launch 및 strided access 비중은 커질 수 있다.

5.2 1차 구현

- destination linear index에서 peer와 peer-local index를 역산하는 one-shot pack kernel.
- recv displacements를 읽어 한 번에 destination 3D layout으로 쓰는 one-shot unpack kernel.
- counts/displacements와 block range는 기존 persistent metadata를 재사용.
- C2J와 J2C를 별도 kernel로 유지해 복잡한 generic index 산술을 피함.
- kernel launch configuration은 128/256 threads를 먼저 비교하고 register spill을 확인.

5.3 2차 후보

후보	기대	위험	착수 조건
tilted transpose	coalesced read/write	shared memory와 edge 처리	Nsight에서 memory inefficiency 확인
fused pack + scale	추가 device pass 제거	수치 순서 변화	해당 scale이 route와 인접할 때
persistent kernel	launch 감소	복잡도/점유율	one-shot launch도 병목일 때

5.4 비교 실험

- backend x pack 조합을 mpi-peer, mpi-one-shot, nccl-peer, nccl-one-shot 네 개로 분리.
- 같은 job allocation 안에서 순서를 교차해 clock/thermal bias를 줄임.
- route detail과 전체 perf를 함께 기록하되 채택은 perf rank-max 중앙값으로 결정.

pack/unpack 비중이 작다면 이 단계는 건너뛰다. 보기 좋은 kernel rewrite보다 통신 단계 수를 줄이는 것이 우선이다.

6. Stage 4 - 복소 TDMA 2-RHS batching

real/imag 두 번 호출을 한 번의 통신 묶음으로 줄이는 구조 변경

6.1 목표

현재 complex TDMA가 real RHS와 imaginary RHS를 독립적으로 core에 전달해 동일한 계수 통신을 반복한다면, 계수 a,b,c는 한 번만 보내고 d를 2-RHS로 묶어 collective 호출 수를 우선 10회에서 5회 수준으로 줄이는 것이 목표다.

6.2 구현 순서

순서	작업	검증
A	현재 PaScaL_TDMA core의 a,b,c,d,solution exchange call count를 debug counter로 확정	호출 수와 bytes 표 작성
B	RHS 차원을 nrhs=2로 확장한 plan/API 설계	nrhs=1 기존 경로 유지
C	a,b,c pack은 1회, d와 solution은 contiguous 2-RHS buffer로 pack	각 phase payload 검증
D	local Thomas update를 RHS loop 또는 vectorized kernel로 확장	real/imag 단독 결과와 비교
E	complex wrapper가 한 번의 core 호출만 수행	final RMS와 perf 중앙값

6.3 수치/메모리 주의

- Thomas algorithm이 coefficient 배열을 in-place로 수정하므로 공유 계수와 RHS별 상태를 명확히 분리.
- 2-RHS layout은 [line, rhs] 연속 배치를 먼저 검토해 coalescing과 NCCL/MPI contiguous payload를 유지.
- temporary memory 증가량을 출력하고 256^3 FP64에서 A100 memory headroom 확인.
- real/imag 재결합 전 각각의 max norm diagnostic을 debug에서만 선택적으로 출력 가능.

6.4 채택 기준

- TDMA collective count가 목표대로 감소.
- RMS 일치, memory leak/반복 solve allocation failure 없음.
- NP=4 이상에서 전체 solve 중앙값 10% 이상 개선 또는 TDMA phase 20% 이상 개선.

7. Stage 5 - TDMA 불변 계수와 reduced factor 캐시

CaNS를 참고하되 불변성이 입증된 데이터만 정상상태에서 재사용

7.1 먼저 분류

데이터	예상 속성	캐시 가능성	필수 확인
grid spacing 기반 a,b,c 원본	격자/BC 고정 시 불변	높음	solve 중 in-place 수정 여부
Fourier eigenvalue 결합 계수	mesh/plan 고정 시 불변	높음	mode별 singular 처리
local elimination 결과	알고리즘이 원본 계수를 수정	조건부	RHS 의존성 분리
reduced boundary factors	계수 고정 시 불변 가능	조건부	rank/communicator shape key
RHS d와 solution	매 solve 변화	불가	2-RHS buffer만 재사용

7.2 cache key

- global/local shape, communicator size/rank ordering, BC type, precision, periodic flag.
- FFT redistribution 이후의 n1td/n2td/n3td와 start/end offsets.
- 계수 생성에 사용한 spacing/eigenvalue version 또는 explicit invalidation counter.

7.3 lifecycle

```
plan_create:
  allocate persistent original coefficients
  build immutable coefficient state
  precompute valid reduced factors

steady_solve:
  copy/reuse only the state modified by Thomas
  communicate RHS and solution

plan_destroy:
  free cache exactly once
```

7.4 실패 방지

- 첫 구현은 cache OFF/ON runtime switch 제공.
- 매 N번째 solve에서 debug-only checksum으로 cached coefficient 상태 검사.
- shape/BC가 변하면 silent reuse 대신 plan invalidate 후 재생성.

최종 목표인 정상상태 TDMA 2회 통신은 2-RHS batching과 factor cache가 모두 성립할 때만 가능하다. 소스 증명 없이 호출 수부터 강제로 줄이면 오답 위험이 가장 크다.

8. Stage 6 - cuDecomp subcommunicator 타당성 시험

이식이 아니라 layout 비용을 포함한 feasibility gate

8.1 왜 후순위인가

CaNS는 cuDecomp가 소유하는 pencil layout과 workspace를 중심으로 설계됐다. 현재 PaScaL은 cube domain decomposition과 자체 I/C/J layout, 자체 TDMA communicator를 이미 가진다. cuDecomp를 route 하나에 넣기 위해 추가 full redistribution이 필요하면 backend 이득보다 layout 변환 비용이 더 커질 수 있다.

8.2 feasibility checklist

질문	통과 조건
현재 C2J/J2C send/recv layout을 cuDecomp pencil descriptor로 직접 표현 가능한가?	추가 full-device reorder 0회
기존 x2 subcommunicator만으로 grid descriptor를 만들 수 있는가?	world 재분해 없이 생성
workspace를 기존 persistent buffer와 공유 가능한가?	추가 peak memory 허용 범위
autotune 비용을 init에만 둘 수 있는가?	steady solve에 autotune/alloc 0회
NCCL direct backend보다 유지보수 이득이 있는가?	성능 동등 이상 + 코드 단순화

8.3 결정

- 모든 조건 통과: isolated prototype branch에서 MPI/NCCL/cuDecomp 3-way 비교.
- 추가 redistribution 1회 이상: 현 route에는 reject, 향후 전체 FFT layout 재설계 때 재검토.
- subcommunicator mapping 불일치: cube decomposition을 바꾸지 않고는 채택하지 않음.

8.4 CaNS autotune 해석

첨부 CaNS input은 is_poisson_pcr_tdma=F이고 1 x 1 구성이다. 그 로그에서 선택된 NCCL backend는 다중 GPU C2J/J2C 성능을 입증하지 않는다. Neuron에서 동일 payload와 communicator로 직접 측정해야 한다.

9. Stage 7-8 - stream/workspace와 local transpose

동기화 삭제보다 dependency 통합이 먼저

9.1 canonical stream

- OpenACC queue 1의 CUDA stream handle을 init에서 얻어 plan에 저장.
- cuFFT plan에 cufftSetStream으로 같은 stream 지정.
- CUDA Fortran kernel launch도 명시적으로 같은 stream을 사용하도록 전환.
- NCCL backend가 있으면 같은 stream에 enqueue.
- CUDA-aware MPI는 stream semantics 지원 여부를 별도 확인. 지원이 없으면 MPI 전 event/sync dependency 유지.

현재처럼 CUDA Fortran kernel, OpenACC, cuFFT, MPI가 섞인 코드에서 일부 라이브러리만 async(1)로 바꾸고 cudaStreamSynchronize를 삭제하면 race가 생길 수 있다. 모든 producer/consumer가 같은 stream 또는 명시적 event로 연결된 뒤에만 sync를 줄인다.

9.2 cuFFT shared workspace

- cufftSetAutoAllocation(plan, 0) 후 각 plan의 workSize 조회.
- 동시에 실행하지 않는 plan들은 max(workSize) 하나의 persistent device workspace 공유.
- plan cache key 변경 시 workspace resize를 init/setup 구간에서만 수행.

9.3 local transpose gate

관측	행동
transpose kernel 합계 < solve의 10%	건너뛸
global load/store efficiency 낮고 합계 >= 10%	shared-memory tiled kernel
transpose와 인접 scale/copy가 함께 큼	fused kernel 후보
kernel은 빠르나 sync가 큼	stream/event dependency부터 수정

9.4 후순위 연구

CUDA Graph, NVSHMEM, cuFFTMp는 위 단계가 끝난 뒤에도 launch 또는 collective가 지배적일 때만 검토한다. 256^3 strong scaling에서는 GPU당 작업량이 빠르게 작아지므로 복잡한 runtime 도입이 항상 이득이라는 보장은 없다.

10. 검증 매트릭스와 변경 통제

한 번에 한 가설만 검증하는 운영 규칙

10.1 필수 실험표

Case	NP	build	반복	저장 결과	목적
S1-debug	1,2,4	debug warmed	1 job 이상	route phase, bytes, BW, RMS	Stage 2 선택
S1-perf	1,2,4	perf	독립 job 3회	rank-max, RMS	새 계측 무영향 확인
Backend	1,2,4,8	perf MPI/NCCL	각 3회	중앙값/min/max	Stage 2 채택
TDMA	1,2,4,8	debug + perf	각 3회	collective count, rank-max	Stage 4/5 채택
Final	1..32	perf	각 3회	scaling table, RMS	최종 보고

10.2 로그 계약

- SLURM_JOB_ID, node list, rank/local-rank/GPU mapping, module list, compiler version.
- git hash 또는 ZIP version, backend switch, warm-up/iteration 수.
- debug log와 perf log 분리. 모든 rank의 solution field 출력 금지.
- RMS는 final iteration에서만 한 번 출력.

10.3 rollback 단위

Stage	runtime switch	rollback 경계
1 route profile	compile-time debug only	performance binary 무변경
2 NCCL	mpi nccl	exchange backend 함수
3 pack	peer oneshot	pack/unpack kernel
4 TDMA batch	nrhs=1 2	complex wrapper/API
5 factor cache	off on	plan lifecycle/cache
6 cuDecomp	mpi nccl cudecomp	prototype branch

두 Stage를 동시에 넣고 빨라졌는지만 보는 방식은 금지한다. RMS 실패나 회귀가 발생하면 해당 Stage의 switch만 OFF로 되돌릴 수 있어야 한다.

11. Neuron 실행 절차

이번 Stage 1 ZIP을 받은 뒤 바로 수행할 명령

11.1 빌드

```
cd /scratch/x3319a08/PaScaL_POISSON_FFT_GPU_dual
module purge
module load nvhpc/25.11_cuda12
module load cuda/12.9.1

make clean
make -j

test -x run/examples_debug.exe
test -x run/examples_perf.exe
```

11.2 NP=1,2,4 순차 검증

```
export PROJECT_ROOT=/scratch/x3319a08/PaScaL_POISSON_FFT_GPU_dual
export POISSON_MPI_SIZES="1 2 4"
export MAX_ACTIVE_JOBS=1
export POISSON_WRITE_SOLUTION=0
export POISSON_DEBUG_COLD_PROFILE=1
export POISSON_DEBUG_WARMUP=5
export POISSON_WARMUP=5
export POISSON_ITERS=100

./submit_all_a100_batched.sh
```

11.3 결과 추출

```
rg -n "FFT route detail\[ROUTE\]|Steady-state average|RMS error" batch_runs/np{1,2,4}/job_*/{debug,perf}/*.log
```

11.4 컴파일 실패 시 우선 확인

- mpif90/mpifort가 NVHPC wrapper인지 which와 --version으로 확인.
- preprocessor가 POISSON_DETAILED_PROFILE을 debug에만 정의하는지 make -n으로 확인.
- CUDA/FFT/MPI library가 CUDA 12.9 계열로 연결되는지 nvfortran -dryrun과 ldd로 기록.
- 소스의 첫 오류만 고친 뒤 clean build. debug와 perf.mod를 섞지 않음.

이 ZIP은 현재 컨테이너에서 전처리/정적 검사까지 완료했지만 NVHPC와 A100이 없어 실제 CUDA Fortran compile/run은 검증하지 못했다. 위 build 결과와 첫 NP=1 debug log를 다음 검토 입력으로 사용한다.

12. 실행 순서와 참고자료

완료 조건을 충족한 Stage만 다음으로 넘긴다

순번	작업	현재 상태	다음 단계 진입 조건
0	측정 harness, dual build, RMS/output 정책	완료	NP=1,2,4 RMS 확인
1	C2J/J2C route 상세 profile	코드 완료	Neuron warmed debug 수집
2	NCCL grouped P2P backend	대기	MPI 비중 gate 통과
3	one-shot pack/unpack	조건부	pack+unpack gate 통과
4	TDMA 2-RHS batching	대기	TDMA call count/비중 확정
5	TDMA factor/cache	대기	불변성 및 in-place 수정 분석 완료
6	cuDecomp feasibility	후순위	추가 redistribution 0회
7	canonical stream + cuFFT workspace	후순위	backend 안정화
8	local tiled/fused transpose	조건부	transpose >= solve 10%
9	Graph/NVSHMEM/cuFFTMp	연구	앞 단계 후에도 명확한 병목

참고자료

- NVIDIA HPC SDK 25.11 Release Notes
- NCCL User Guide - Point-to-point communication
- cuFFT Documentation - plan work area and stream APIs
- 첨부 CaNS source commit f3019f93ebac060ca2bbca93aab9406cc8844d7a.
- 이번 구현본 STAGE1_ROUTE_PROFILE.md와 README_DUAL_PROFILE.md.

다음 즉시 행동: 새 ZIP을 Neuron에서 clean build하고 NP=1,2,4를 순차 실행한다. warmed debug의 [ROUTE] 12개 수치와 perf rank-max/RMS를 모은 뒤 Stage 2를 NCCL로 갈지, Stage 3 pack rewrite를 먼저 할지 결정한다.

문서 성격: source-level work order. 성능 예상치는 보장이 아니라 구현 우선순위 가설이며, 최종 판정은 동일 정확도와 Neuron 실측으로 확정한다.